

6교시: 2주차 발표

수업 목표

- Docker 실행 방법을 기능 소개가 아니라 evidence 중심으로 발표한다.
- 겪은 장애, 해결 과정, 남은 위험을 짧고 명확하게 공유한다.
- peer feedback을 통해 README와 실행 절차를 개선한다.

50분 흐름

시간	활동	비중	산출
0-8분	발표 기준 안내	설명 15%	presentation rubric
8-18분	발표 카드 작성	실행 20%	3-minute card
18-38분	학생 발표	발표 40%	peer feedback
38-46분	evidence gap 표시	실행 15%	patch list
46-50분	다음 교시 Q&A 연결	설명 10%	question list

Visual 1: evidence 중심 발표 구조

2주차 발표 구조

3분 · 증거 중심 발표

- 0:00 **app** 프로젝트 개요 및 목표
- 0:20 **build** Dockerfile 빌드 증거
- 0:40 **run** 컨테이너 실행 증거
- 1:10 **check** 서비스/DB 확인 증거
- 1:40 **failure / RCA** 문제 발생 및 원인 분석
- 2:10 **security** 보안 점검 및 조치
- 2:30 **cleanup** 정리 및 재현 가능 상태
- 2:50 **Week 3 질문** 다음 단계 질문 및 연결
- 3:00 **마무리**

app (0:00~0:20)

- 무엇을 만들었나?
- 핵심 기능
- 구성도

build (0:20~0:40)

```
$ docker build -t myapp:1.0 .
...
[internal] load build definition
...
[internal] load dockerignore
...
[1/3] FROM nginx:1.25-alpine
...
[5/5] exporting to image
...
naming to docker.io/library/myapp:1.0
...
digest: sha256:8ad312...
```

run (0:40~1:10)

```
$ docker run -d --name myapp -p 8080:80 myapp:1.0
c0ffee9b7e2f4c1a9d2e...
$ docker ps
CONTAINER ID   IMAGE     PORTS          STATUS
c0ffee9b7e2f4   myapp:1.0  0.0.0.0:8080->80/tcp   Up 3s
```

check (1:10~1:40)

HTTP 200

```
$ docker logs myapp
/docker-entrypoint.sh: /docker-entrypoint.d/
/start...
nginx: [notice] start worker processes
nginx: [notice] ready to serve
```

failure / RCA (1:40~2:10)

증상: 연결 안 됨
원인: 포트 오라
조치: 8080 → 80
결과: 정상

```
$ docker ps
... 0.0.0.0:8080->80/tcp
$ curl -I http://localhost:8080
HTTP/1.1 200 OK
```

security (2:10~2:30)

```
$ docker scout cves myapp:1.0
✓ No critical vulnerabilities
$ hadolint Dockerfile
✓ No issues found
$ docker run --read-only --cap-drop ALL myapp:1.0
✓ OK
```

cleanup (2:30~2:50)

```
$ docker stop myapp
$ docker rm myapp
$ docker volume prune -f
$ docker system prune -f
```

Week 3 질문 (2:50~3:00)

- DB 데이터 영속화는 어떻게?
- 환경별 설정은 어떻게 관리할까?
- 테스트 자동화는 어떻게 할까?

★ 핵심은 “증거”입니다!

짧게! 명확하게! 증거로!

CHECKLIST

- ✓ build
- ✓ check
- ✓ 123cA

이 visual은 발표가 기능 소개가 아니라 problem, run, evidence, failure, risk, next step으로 구성되어야 함을 보여준다.

핵심 설명

현업 발표는 "만든 것 자랑"이 아니라 의사결정자가 다음 행동을 할 수 있게 증거를 전달하는 일이다. Week 2 발표도 마찬가지다.

좋은 발표는 다음 질문에 답한다.

1. 무엇을 Docker로 실행했는가?
2. 어떤 명령으로 build/run/check하는가?

3. 정상 상태 evidence는 무엇인가?
4. 어떤 장애를 겪었고 어떻게 재확인했는가?
5. 남은 위험과 다음 주차 질문은 무엇인가?

3분 발표 카드

Week 2 Docker Presentation

- App:
- Build command:
- Run command:
- Compose command:
- HTTP evidence:
- Failure/RCA:
- Security note:
- Cleanup:
- Week 3 question:

발표 시간 배분

구간	내용
0:00-0:30	앱과 Docker화 목표
0:30-1:10	build/run/compose 명령
1:10-1:50	HTTP/log/status evidence
1:50-2:30	장애/RCA
2:30-3:00	보안/cleanup/Week 3 질문

좋은 발표와 약한 발표

유형	특징
약한 발표	"Docker로 실행했습니다" 중심
중간 발표	명령은 있으나 expected output 부족
좋은 발표	명령, 상태, HTTP, RCA, risk, next step 포함

peer feedback form

Peer Feedback

- 가장 재현 가능했던 설명:
- 누락된 evidence:
- 보안/cleanup 위험:
- README 개선 제안:
- Week 3 연결 질문:

학술 기준 연결

ABET의 커뮤니케이션 outcome과 AAC&U oral communication rubric 관점이 적용된다. 기술 발표는 청중이 이해하고 재현하고 판단할 수 있게 구성되어야 한다.

실무 insight

DevOps 회의에서 "성공했습니다"보다 중요한 것은 evidence와 residual risk다. 특히 배포 전 공유에서는 정상 확인 방법과 rollback/cleanup 방법이 빠지면 의사결정이 어렵다.

오해 점검

오해	교정
발표는 결과만 보여주면 된다	재현 가능한 과정과 증거가 필요하다
실패 이야기는 빠는 게 좋다	RCA는 운영 성숙도의 증거다
시간이 짧으면 보안은 생략한다	secret/cleanup 위험은 짧게라도 말한다
화면이 보이면 evidence다	command/output/path가 같이 있어야 한다

평가 기준

기준	2점 evidence
구조	3분 카드 기준을 지켰다
evidence	HTTP/status/log/tag를 제시했다
RCA	실패와 재확인을 설명했다
책임	보안/cleanup 위험을 언급했다
연결	Week 3 질문을 제시했다

전이 과제

발표 후 받은 feedback 중 하나를 README에 반영하고 어떤 문장을 바꿨는지 기록한다.

공식 근거 링크

- AAC&U VALUE Rubrics: <https://www.aacu.org/initiatives/value-initiative/value-rubrics>
- Google SRE Postmortem Culture: <https://sre.google/sre-book/postmortem-culture/>

발표 평가 루브릭

항목	0점	1점	2점
문제/목표	없음	앱 설명만	Docker화 목표 명확
실행	없음	명령 일부	build/run/compose 명령
증거	없음	화면만	HTTP/log/status/tag
장애	없음	증상만	RCA와 recheck
보안	없음	주의만	secret/push/cleanup 기준
다음 연결	없음	막연함	Week 3 질문 구체

발표 후 기록

Presentation Result

- Presented by:
- Strongest evidence:
- Missing evidence:
- Peer question:
- Patch action:
- Recheck:

발표 금지 습관

습관	이유
"아무튼 됐습니다"	평가 가능한 evidence 없음
token이 보이는 화면 공유	credential leak
실패를 생략	운영 학습 기회 상실
README 없이 구두 설명만	handoff 불가
port와 path를 말하지 않음	재현 불가

좋은 발표 문장

이 앱은 `paperclip/week2-day5-integration:local` tag로 build했고, `localhost:18085`에서 HTTP 200과 `week2-day5-integration-v1` body marker를 확인했습니다. wrong port 실패를 재현했고, README에 host port와 cleanup을 추가했습니다.

peer 질문 예시

1. 이 image tag를 다른 사람이 어떻게 재현하나요?
2. README만 보고 cleanup할 수 있나요?
3. public push를 해도 안전한 상태인가요?
4. 장애가 다시 나면 어디를 먼저 보나요?
5. Week 3에서 service가 늘어나면 이 문서는 어떻게 바뀌나요?

발표 전 self-check

- ☐ 발표 시간이 3분을 넘지 않는다.
- ☐ build 명령을 말한다.
- ☐ run 또는 Compose 명령을 말한다.
- ☐ HTTP evidence를 말한다.
- ☐ 장애/RCA를 하나 말한다.
- ☐ secret/push/cleanup 위험을 말한다.
- ☐ Week 3 질문으로 마무리한다.

Lesson 6 Exit Ticket

Exit Ticket

- 내 발표에서 가장 강한 evidence:
- 가장 약한 evidence:
- 받은 질문:
- README에 반영할 수정:

발표 마감 기준

발표가 끝났을 때 학생은 "무엇을 보여줬는가"보다 "무엇이 재현 가능한가"를 기준으로 자기 발표를 평가한다. 발표 중 보여준 화면과 README에 남은 명령이 서로 일치해야 한다.

마감 확인:

- 발표에서 말한 port가 README port와 같은가?
- 발표에서 말한 tag가 실제 image tag와 같은가?
- 발표에서 말한 cleanup 명령이 실제로 실행 가능한가?

발표 평가 루브릭

항목	0점	1점	2점
문제/목표	없음	앱 설명만	Docker화 목표 명확
실행	없음	명령 일부	build/run/compose 명령
증거	없음	화면만	HTTP/log/status/tag
장애	없음	증상만	RCA와 recheck
보안	없음	주의만	secret/push/cleanup 기준
다음 연결	없음	막연함	Week 3 질문 구체

발표 금지 습관

습관	이유
"아무튼 됐습니다"	평가 가능한 evidence 없음
token이 보이는 화면 공유	credential leak
실패를 생략	운영 학습 기회 상실
README 없이 구두 설명만	handoff 불가
port와 path를 말하지 않음	재현 불가

좋은 발표 문장

이 앱은 `paperclip/week2-day5-integration:local` tag로 build했고, `localhost:18085`에서 HTTP 200과 `week2-day5-integration-v1` body marker를 확인했습니다. wrong port 실패를 재현했고, README에 host port와 cleanup을 추가했습니다.

peer 질문 예시

1. 이 image tag를 다른 사람이 어떻게 재현하나요?
2. README만 보고 cleanup할 수 있나요?
3. public push를 해도 안전한 상태인가요?
4. 장애가 다시 나면 어디를 먼저 보나요?
5. Week 3에서 service가 늘어나면 이 문서는 어떻게 바뀌나요?